

Laboratorul 1 - Android Studio, Git si GitHub

1. Obiectiv

La finalul laboratorului trebuie sa aveti:

- Android Studio instalat;
- Android SDK configurat;
- un emulator Android functional;
- primul proiect creat folosind Java si XML;
- aplicatia rulata pe emulator sau pe un telefon Android;
- Git instalat si identitatea Git configurata;
- un cont GitHub cu un username profesional;
- repository-ul privat `nume-prenume-labS0` creat pe GitHub;
- proiectul Android publicat in repository;
- minimum doua commit-uri relevante si un fisier `README.md`;
- commit-urile trimise pe GitHub prin `push`;
- contul `alexandra-luca-professor` adaugat drept colaborator;
- linkul repository-ului pregatit pentru predare.

Descarcati Android Studio numai de pe [pagina oficiala Android Developers](#).

Pentru acest laborator nu este necesara instalarea separata a unui JDK, Gradle sau Maven. Android Studio include mediul Java necesar, iar proiectul descarca automat versiunea Gradle de care are nevoie.

2. Instalarea pe Windows

Pasul 1 - Verificarea calculatorului

Inainte de instalare, verificati ca aveti:

- Windows 10 sau Windows 11 pe 64 de biti;
- suficient spatiu liber pentru Android Studio, Android SDK si emulator;
- o conexiune stabila la internet;
- virtualizarea procesorului activata.

Pentru verificarea virtualizarii:

1. Deschideti Task Manager.

2. Selectati fila Performance.
3. Selectati CPU.
4. Verificati daca apare:

Virtualization: Enabled

Virtualizarea permite procesorului sa ruleze eficient dispozitive virtuale. Android Emulator foloseste aceasta functie pentru a simula un telefon. Daca apare **Disabled**, emulatorul poate functiona foarte greu sau poate sa nu porneasca. Activarea se face din BIOS/UEFI, iar pasii difera in functie de producatorul calculatorului.

Pasul 2 - Descarcarea Android Studio

1. Accesati [pagina oficiala Android Studio](#).
2. Pagina detecteaza, de regula, sistemul de operare.
3. Selectati varianta recomandata Windows (64-bit) cu installer .exe.
4. Acceptati termenii si conditiile.
5. Descarcati fisierul de instalare.

Nu descarcati Android Studio de pe site-uri terte.

Pasul 3 - Rularea programului de instalare

1. Deschideti fisierul .exe descarcata.
2. Daca Windows afiseaza o confirmare, selectati Yes.
3. Apasati Next.
4. Pastrati selectate componentele:

Android Studio
Android Virtual Device

5. Pastrati directorul implicit de instalare.
6. Apasati Install.
7. Asteptati finalizarea instalarii.
8. Selectati Start Android Studio.
9. Apasati Finish.

Pasul 4 - Configurarea initiala

La prima pornire, Android Studio poate intrea daca doriti sa importati configurari dintr-o versiune anterioara. Daca este prima instalare si apare aceasta optiune, selectati:

Do not import settings

In programul de configurare:

1. Selectati configurarea Standard, daca apare aceasta optiune.
2. Alegeti tema vizuala dorita.
3. Verificati componentele propuse pentru instalare.
4. Acceptati licentele Android SDK.
5. Apasati Finish.
6. Asteptati descarcarea si instalarea componentelor.

Nu inchideti Android Studio in timpul descarcarii SDK-ului.

3. Instalarea pe macOS

Pasul 1 - Verificarea tipului de procesor

1. Deschideti meniul Apple.
2. Selectati About This Mac.
3. Verificati informatia afisata la Chip sau Processor.

Puteti avea:

```
Apple M1, M2, M3, M4 sau mai nou -> Apple Silicon  
Intel Core i5, i7 etc.           -> Intel
```

Este important sa descarcati varianta Android Studio corespunzatoare procesorului. Suportul pentru Mac-urile Intel este in curs de eliminare si poate disparea in versiuni viitoare.

Pasul 2 - Descarcarea Android Studio

1. Accesati [pagina oficiala Android Studio](#).
2. Derulati pana la sectiunea Android Studio downloads, daca butonul principal nu afiseaza varianta dorita.
3. Alegeti versiunea corespunzatoare:

```
Mac (64-bit, ARM) -> Apple Silicon  
Mac (64-bit)      -> Intel
```

4. Acceptati termenii si conditiile.
5. Descarcati fisierul .dmg.

Pasul 3 - Instalarea aplicatiei

1. Deschideti fisierul .dmg.
2. Trageti pictograma Android Studio in directorul Applications.
3. Deschideti directorul Applications.
4. Porniti Android Studio.
5. Confirmati deschiderea aplicatiei daca macOS afiseaza un avertisment.

Daca macOS blocheaza pornirea:

1. Deschideti System Settings.
2. Accesati Privacy & Security.
3. Gasiti mesajul referitor la Android Studio.
4. Selectati Open Anyway, numai daca aplicatia a fost descarcata de pe site-ul oficial.

Pasul 4 - Configurarea initiala

Daca este prima instalare si apare optiunea, selectati:

```
Do not import settings
```

In programul de configurare:

1. Selectati configurarea Standard, daca apare aceasta optiune.
2. Alegeti tema vizuala dorita.
3. Acceptati licentele Android SDK.
4. Apasati Finish.

5. Asteptati instalarea componentelor.

4. Crearea primului proiect Java si XML

Pasii urmasori sunt aceiasi pe Windows si macOS.

Pasul 1 - Crearea proiectului

1. Din ecranul principal Android Studio, apasati New Project.
2. Selectati categoria Phone and large screens. In unele versiuni mai vechi, aceasta categorie poate aparea ca Phone and Tablet.
3. Derulati lista de template-uri, daca este necesar, si alegeti Empty Views Activity. Nu alegeti Empty Activity cu pictograma Jetpack Compose.
4. Apasati Next.

Empty Activity si Empty Views Activity sunt template-uri diferite. Pentru laborator alegeti Empty Views Activity, deoarece acesta permite selectarea limbajului Java si creeaza un fisier de layout XML.

Pasul 2 - Configurarea proiectului

Completati campurile astfel:

```
Name: Lab01HelloAndroid
Package name: ro.so.android.numeprenume
Language: Java
Minimum SDK: API 26
```

Inlocuiti numeprenume cu numele vostru, scris fara spatii si fara diacritice.

La Build configuration language, pastrati optiunea implicita Kotlin DSL (`build.gradle.kts`) [Recommended]. Aceasta este folosita numai pentru fisierele de configurare si compilare ale proiectului. Codul aplicatiei ramane scris in Java, iar interfata este definita in XML.

5. Selectati directorul in care va fi salvat proiectul.
6. Apasati Finish.
7. Asteptati deschiderea proiectului si finalizarea operatiei Gradle Sync.

Nu incercati sa rulati proiectul inainte ca sincronizarea Gradle sa se termine.

5. Verificarea Android SDK

Dupa deschiderea proiectului:

1. Apasati pictograma Settings (rotita) din dreapta sus.
2. Selectati SDK Manager.

In unele versiuni sau configurari ale interfetei, SDK Manager poate fi deschis si din Tools -> SDK Manager. O alta varianta este Find Action: apasati Ctrl+Shift+A pe Windows/Linux sau Command+Shift+A pe macOS, apoi cautati SDK Manager.

3. In SDK Platforms, instalati platforma ceruta de compileSdk din app/build.gradle.kts.
4. In fila SDK Tools, verificati componentele:

```
Android SDK Build-Tools
Android SDK Platform-Tools
Android SDK Command-line Tools (latest)
Android Emulator
```

5. Bifati componentele care lipsesc.
6. Apasati Apply, apoi OK.
7. Acceptati licentele.
8. Asteptati finalizarea instalarii.

Daca doriti sa vedeti versiunile individuale ale componentelor, bifati Show Package Details.

6. Crearea emulatorului

Android Studio foloseste un Android Virtual Device - AVD - pentru a simula un telefon Android. Consultati si [ghidul oficial pentru dispozitive virtuale](#).

Pasul 1 - Deschiderea Device Manager

1. Din proiectul deschis, apasati pictograma Device Manager din bara verticala din dreapta.
2. Daca pictograma nu este vizibila, selectati Tools -> Device Manager.
3. In panoul Device Manager, apasati Create Virtual Device sau simbolul +.

Puteti folosi si Find Action: Ctrl+Shift+A pe Windows/Linux sau Command+Shift+A pe macOS, apoi cautati Device Manager.

Pasul 2 - Alegerea telefonului virtual

1. Selectati categoria Phone.
2. Alegeti un dispozitiv Pixel standard.
3. Apasati Next, daca apare. In versiunile noi, Android Studio poate deschide direct ecranul Configure virtual device.

Pasul 3 - Alegerea imaginii Android

Imaginea de sistem reprezinta versiunea sistemului Android care va rula in telefonul virtual.

1. Selectati o versiune Android stabila recomandata de Android Studio, marcata de regula cu o stea.
2. Pentru un emulator obisnuit, pastrati serviciile Google Play Store, daca sunt disponibile.
3. Daca imaginea nu este instalata, apasati pictograma de descarcare. Descarcarea poate avea aproximativ 1-2 GB.
4. Asteptati finalizarea descarcarii.
5. Selectati imaginea si apasati Next sau Finish, in functie de versiunea Android Studio.

Pe calculatoarele cu Apple Silicon trebuie folosita o imagine compatibila ARM. Pe sistemele Intel/AMD folositi imaginea recomandata de Android Studio pentru platforma respectiva.

Pasul 4 - Finalizarea configurarii

1. Pastrati setarile recomandate.
2. Dati emulatorului un nume usor de recunoscut.
3. Apasati Finish.
4. Porniti emulatorul din Device Manager.
5. Asteptati aparitia ecranului principal Android.

Prima pornire poate dura mai mult.

7. Structura proiectului

In panoul Project identificati urmatoarele elemente:

app/src/main/java/	-> clasele Java
app/src/main/res/layout/	-> interfetele XML
app/src/main/res/drawable/	-> imagini si forme
app/src/main/res/mipmap/	-> pictograma aplicatiei
app/src/main/res/values/	-> texte, culori si teme
app/src/main/AndroidManifest.xml	-> activitati si permisiuni

Clasa principala este, de regula:

```
MainActivity.java
```

Interfata principala este, de regula:

```
activity_main.xml
```

In acest laborator trebuie doar sa identificati fisierele. Ele vor fi explicate in detaliu in laboratoarele urmatoare.

8. Rularea aplicatiei

1. Porniti emulatorul.
2. Asteptati pana cand pe ecranul telefonului virtual apare interfata Android.
3. In bara de sus din Android Studio, verificati ca este selectata configuratia app.
4. Selectati emulatorul pornit din lista dispozitivelor.
5. Apasati butonul verde Run 'app' sau folositi Shift+F10 pe Windows/Linux, respectiv Control+R pe macOS.
6. Asteptati compilarea, instalarea si pornirea aplicatiei.
7. Verificati ca ecranul definit de `MainActivity` apare in emulator.

Nu se ruleaza direct fisierul `MainActivity.java`. Se ruleaza modulul app, iar Android deschide automat `MainActivity`, deoarece aceasta este configurata ca activitate de lansare in `AndroidManifest.xml`.

La prima rulare, Gradle poate descarca dependinte, iar procesul poate dura mai mult.

9. Rularea pe un telefon Android personal

Alegeti un telefon cu Android 8.0 (API 26) sau mai nou. Consultati si ghidul oficial.

Pe telefon:

1. Deschideti Settings.
2. Accesati About phone.
3. Apasati de mai multe ori pe Build number, pana se activeaza Developer Options.
4. Deschideti Developer Options.
5. Activati USB debugging.
6. Conectati telefonul la calculator.
7. Acceptati autorizarea afisata pe telefon.

Pe Windows poate fi necesar un driver USB furnizat de producatorul telefonului.

Telefonul ar trebui sa apara in lista dispozitivelor din Android Studio. Selectati-l si apasati Run.

10. Probleme frecvente

Gradle Sync nu se finalizeaza

Verificati:

- conexiunea la internet;
- daca licentele SDK au fost acceptate;
- daca Android SDK este instalat;
- mesajul complet afisat in fereastra Build.

Emulatorul nu porneste

Verificati:

- daca virtualizarea este activata;
- daca imaginea Android este instalata;
- daca exista suficienta memorie disponibila;
- daca a fost aleasa arhitectura corecta pentru procesor.

Dispozitivul nu apare

Verificati:

- daca emulatorul este pornit;
- daca USB debugging este activat;
- daca ati acceptat autorizarea pe telefon;
- cablul USB;
- driverul dispozitivului, pe Windows.

Nu apare activity_main.xml

Cel mai probabil a fost ales un template Jetpack Compose. Creati un proiect nou si selectati Empty Views Activity, cu limbajul Java.

Aplicatia nu compileaza

Verificati mesajul complet din fereastra Build, apoi incercati:

```
File -> Sync Project with Gradle Files
```

Nu reinstalati Android Studio inainte de a identifica mesajul concret al erorii.

11. Checklist pentru Partea I - Android Studio

```
[ ] Android Studio este instalat
[ ] Android SDK este configurat
[ ] Android Emulator este instalat
[ ] Emulatorul porneste
[ ] Proiectul foloseste Java
[ ] Proiectul contine activity_main.xml
[ ] Gradle Sync se finalizeaza fara erori
[ ] Aplicatia compileaza
[ ] Aplicatia ruleaza pe emulator sau telefon
```

Dupa finalizarea acestei parti continuati cu Partea a II-a: instalarea Git, crearea contului GitHub si publicarea proiectului in repository-ul de laboratoare.

Partea a II-a - Git si GitHub

12. Ce sunt Git si GitHub?

Git este un sistem de control al versiunilor instalat pe calculator. El pastreaza istoricul modificarilor unui proiect si permite revenirea la versiuni anterioare.

GitHub este o platforma online care gazduieste repository-uri Git. GitHub permite pastrarea unei copii online a proiectului, colaborarea si accesul cadrului didactic la cod.

Git	GitHub
Program instalat local	Platforma online
Functioneaza si fara internet	Necesita internet pentru sincronizare
Creeaza istoricul prin commit-uri	Gazduieste repository-ul si commit-urile
Urmareste modificarile fisierelor	Permite partajarea si colaborarea

Git si GitHub nu sunt acelasi lucru. Puteti folosi Git fara GitHub, dar pentru predarea laboratoarelor vom folosi ambele.

13. De ce folosim Git?

Git este util deoarece:

- pastreaza istoricul proiectului;
- arata ce fisiere au fost modificate;
- permite revenirea la o versiune functionala;
- reduce riscul pierderii codului;
- demonstreaza progresul realizat pe parcurs;
- este folosit in majoritatea proiectelor software profesionale.

Un commit trebuie privit ca o salvare logica a unei etape, nu ca o simpla apasare a butonului Save.

Exemple de mesaje bune:

```
Lab 01 - creare proiect Android
Lab 01 - adaugare README
Lab 02 - adaugare ecran de rezultat
Fix - corectare inchidere aplicatie
```

Exemple de mesaje nerecomandate:

```
update
ceva
final
final2
merge
```

14. Vocabular Git esential

- Repository - proiectul impreuna cu istoricul sau Git.

- Working directory - fisierele proiectului la care lucrați.
- Staging area - modificările selectate pentru următorul commit.
- Commit - o versiune salvată în istoricul local.
- Branch - o linie separată de dezvoltare a proiectului. Branch-urile permit lucrul la o funcționalitate sau la un experiment fără modificarea imediată a versiunii principale. Branch-ul principal se numește, de regulă, `main`.
- Remote - adresa copiei online a repository-ului.
- Origin - numele folosit, de regulă, pentru repository-ul remote principal.
- Push - trimite commit-urile locale pe GitHub.
- Pull - aduce modificările de pe GitHub pe calculator.
- Clone - descarcă un repository existent pe un calculator.
- `.gitignore` - indică fișierele care nu trebuie incluse în repository.

Fluxul obișnuit este:

```
Modificati fisierele
      ↓
Verificati modificarile
      ↓
Selectati fisierele pentru commit
      ↓
Creati commit-ul
      ↓
Trimiteti commit-ul pe GitHub prin push
```

15. Instalarea Git pe Windows

1. Accesați [pagina oficială Git](#).
2. Descărcați versiunea pentru Windows pe 64 de biți.
3. Porniți installerul.
4. Pastrați opțiunile implicite recomandate.
5. Asigurați-vă că Git este disponibil din linia de comandă.
6. Finalizați instalarea.
7. Închideți și redeschideți Android Studio dacă acesta era pornit.

Deschideți terminalul din Android Studio folosind una dintre variante:

- apăsați meniul principal (pictograma cu trei linii din colțul stâng sus), apoi View -> Tool Windows -> Terminal;
- apăsați pictograma Terminal din bara cu ferestre auxiliare, dacă este vizibilă;
- folosiți scurtătura `Alt+F12` pe Windows/Linux sau `Option+F12` pe macOS.

Verificați instalarea:

```
git --version
```

Rezultatul trebuie să conțină numărul versiunii Git instalate.

16. Instalarea Git pe macOS

Deschideți aplicația Terminal și executați:

```
git --version
```

Daca Git este instalat, va fi afisata versiunea. Daca macOS solicita instalarea Command Line Developer Tools, acceptati instalarea si asteptati finalizarea.

Git poate fi descarcat si de pe [pagina oficiala pentru macOS](#).

Dupa instalare, inchideti si redeschideti Android Studio, apoi verificati din nou:

```
git --version
```

17. Crearea contului GitHub

1. Accesati [GitHub](#).
2. Selectati Sign up.
3. Creati un cont personal.
4. Pentru laborator, folositi obligatoriu formatul numeprenume.
Exemplu: Popescu Andrei -> popescuandrei.
Scrieti cu litere mici, fara spatii, diacritice, puncte, cratime sau underscore (_).
Daca username-ul este ocupat, adaugati un numar la final: popescuandrei2.
Pentru nume sau prenume compuse, uniti toate componentele, fara separatori.
In README treceti numele complet, grupa si username-ul folosit.

5. Confirmati adresa de e-mail.

6. Pastrati in siguranta datele de recuperare ale contului.

Autentificarea cu doi factori este recomandata. Nu transmiteti altor persoane parola, codurile de autentificare sau token-urile contului.

Retineti username-ul si adresa de e-mail asociata contului. Adresa va fi folosita la configurarea Git.

18. Configurarea identitatii Git

Acesta este primul pas de configurare realizat din terminal. Deschideti terminalul integrat in Android Studio cu Alt+F12 pe Windows/Linux sau Option+F12 pe macOS.

Fiecare commit retine numele si adresa de e-mail ale autorului. Executati o singura data pe calculator:

```
git config --global user.name "Nume Prenume"  
git config --global user.email "adresa-folosita-pe-github"
```

Folositi numele real si adresa de e-mail asociata contului GitHub. Nu introduceti username-ul GitHub in locul numelui real. Pentru a nu expune adresa personala, folositi adresa noreply de pe GitHub.

Verificati configurarea:

```
git config --global --list
```

Aceste comenzi configureaza autorul commit-urilor. Nu creeaza repository-ul si nu publica proiectul.

19. Metoda recomandata la laborator - comenzile din terminal

La laborator vom folosi comenzile Git din terminalul integrat in Android Studio. Astfel vedeti explicit fiecare operatie realizata de Git si puteti folosi aceleasi comenzi si in alte editoare sau medii de dezvoltare.

Interfata Android Studio ramane disponibila si este prezentata separat in sectiunea 20. Pentru acelasi pas folositi fie comanda, fie butonul echivalent, nu ambele.

Pasul 1 - Deschiderea terminalului proiectului

1. Deschideti proiectul Lab01HelloAndroid.

2. Apasati Alt+F12 pe Windows/Linux sau Option+F12 pe macOS.
3. Verificati ca terminalul este deschis in directorul proiectului Lab01HelloAndroid.
4. Verificati Git:

```
git --version
```

Pasul 2 - Crearea repository-ului Git local

Executati o singura data in proiect:

```
git init
```

Verificati starea:

```
git status
```

`git init` activeaza Git numai in proiectul local. Nu creeaza inca repository-ul pe GitHub.

Pasul 3 - Verificarea fisierului .gitignore

.gitignore exclude fisierele neurmarite inca de Git; nu elimina automat fisierele deja adaugate in staging sau in commit-uri. Verificati continutul sau inainte de git add.

Nu trebuie publicate:

```
local.properties
.idea/
.gradle/
build/
app/build/
*.iml
*.jks
*.keystore
```

Nu publicati niciodata parole, token-uri, chei API sau chei de semnare.

Daca apar fisiere nedorite, completati .gitignore. Daca sunt deja in staging, refaceti selectia conform pasului 4. Daca sunt deja in commit-uri, cereti ajutor inainte de push.

Pentru primul commit trebuie incluse fisierele care permit deschiderea, compilarea si intelegerea proiectului:

```
.gitignore
build.gradle.kts
settings.gradle.kts
gradle.properties
gradlew
gradlew.bat
gradle/libs.versions.toml          -> daca exista
gradle/wrapper/gradle-wrapper.jar
gradle/wrapper/gradle-wrapper.properties
app/.gitignore                    -> daca exista
app/build.gradle.kts
app/proguard-rules.pro
app/src/main/AndroidManifest.xml
app/src/main/java/.../MainActivity.java
app/src/main/res/layout/activity_main.xml
app/src/main/res/drawable/...
app/src/main/res/mipmap-*/...
app/src/main/res/values/...
app/src/test/...                  -> daca exista
app/src/androidTest/...           -> daca exista
```

Secventa ... indica directoarele intermediare sau fisierele generate de template. De exemplu, calea completa pentru MainActivity.java depinde de package name-ul ales.

Daca proiectul foloseste fisiere Gradle cu extensia .gradle in loc de .gradle.kts, includeti fisierele existente in proiect. In ghidul nostru folosim varianta implicita Kotlin DSL, cu extensia .kts.

Fisierele `gradlew`, `gradlew.bat` si continutul directorului `gradle/wrapper` trebuie incluse. Acestea permit deschiderea si compilarea proiectului cu versiunea Gradle configurata, fara instalarea separata a Gradle. Fisierul `local.properties` nu se include deoarece contine configurarea locala si calea SDK-ului de pe calculatorul vostru.

Pasul 4 - Selectarea fisierelor pentru commit

Mai intai afisati fisierele modificate:

```
git status
```

`git status` nu adauga si nu modifica fisiere. Comanda doar arata ce fisiere sunt noi, modificate sau deja selectate pentru commit.

Priviti lista afisata, apoi alegeti o singura varianta dintre cele de mai jos.

Varianta A - adaugarea tuturor modificarilor

Daca toate fisierele afisate sunt necesare proiectului si `.gitignore` este corect, executati:

```
git add .
```

Punctul `.` inseamna „toate modificarile neignorare din directorul curent si din subdirectoarele sale”. Comanda selecteaza toate fisierele vizibile pentru Git, cu exceptia celor excluse prin `.gitignore`.

Pentru primul commit, aceasta este varianta cea mai simpla, dar trebuie folosita numai dupa verificarea listei cu `git status`.

Varianta B - adaugarea numai a anumitor fisiere

Daca doriti sa includeti doar anumite fisiere, scrieti numele sau calea fiecaruia:

```
git add fisier1 fisier2
```

Exemplu:

```
git add README.md app/src/main/AndroidManifest.xml
```

Aceasta comanda selecteaza numai `README.md` si `AndroidManifest.xml`. Celelalte modificari raman neincluse in commit.

Daca doriti sa selectati explicit toate componentele principale ale proiectului pentru primul commit, puteti folosi:

```
git add .gitignore build.gradle.kts settings.gradle.kts gradle.properties gradlew gradlew.bat gradle app
```

`git add fisier1 fisier2` nu anuleaza o selectie anterioara facuta cu `git add .`

Pentru a reface selectia, lucrati din radacina proiectului:

- inainte de primul commit: `git rm -r --cached .`

- daca exista deja un commit: `git restore --staged .`

Fisierele raman pe disc. Verificati `git status`, apoi adaugati fisierele dorite.

Dupa varianta aleasa, verificati din nou:

```
git status
```

Fisierele selectate pentru commit apar acum in sectiunea `Changes to be committed`.

Pentru primul commit, verificati ca apar cel putin fisierele Gradle, directorul `gradle`, directorul `app`, `MainActivity.java`, `activity_main.xml` si `AndroidManifest.xml`. Verificati si ca `local.properties`, `.idea`, `.gradle` si directoarele `build` nu apar in lista.

Pasul 5 - Crearea primului commit

```
git commit -m "Lab 01 - proiect Android initial"
```

Commit-ul este o versiune salvata in istoricul local. Proiectul nu a fost inca trimis pe GitHub.

Pasul 6 - Crearea repository-ului pe GitHub

1. Deschideti [GitHub](#) si autentificati-va.
2. Apasati simbolul + din partea de sus.
3. Selectati New repository.
4. Introduceti numele:

```
nume-prenume-lab50
```

5. Introduceti descrierea:
Laboratoare Sisteme de Operare - Android
6. Selectati Private.
7. Nu bifati adaugarea unui README, a unui .gitignore sau a unei licente. Aceste fisiere exista sau vor fi create local.
8. Apasati Create repository.

Pasul 7 - Conectarea proiectului local la GitHub

Imediat dupa crearea repository-ului, GitHub afiseaza zona Quick setup.

1. In zona Quick setup, selectati fila HTTPS, nu SSH.
2. In campul alaturat apare linkul repository-ului. de forma:

```
https://github.com/USERNAME/nume-prenume-lab50.git
```

3. Apasati pictograma Copy din partea dreapta a campului. Linkul este copiat in clipboard.
4. Reveniti in terminalul Android Studio.

Un branch este o linie de dezvoltare a proiectului. Echipele folosesc branch-uri separate pentru functionalitati noi, corectarea erorilor sau experimente, astfel incat versiunea principala sa ramana stabila. In acest laborator lucram doar pe branch-ul principal, numit `main`. Branch-uri suplimentare vor putea fi introduse ulterior.

5. Executati prima comanda:

```
git branch -M main
```

6. Scrieti `git remote add origin`, adaugati un spatiu si lipiti linkul copiat folosind Ctrl+V pe Windows/Linux sau Command+V pe macOS:

```
git remote add origin LINKUL_COPIAT
```

Exemplu:

```
git remote add origin https://github.com/popescuandrei/popescu-andrei-lab50.git
```

7. Trimiteți primul commit pe GitHub:

```
git push -u origin main
```

Nu copiat linkul din exemplul ghidului. Folositi linkul HTTPS afisat in propriul repository.

Semnificatia comenzilor:

```
git branch -M main      -> redenumeste branch-ul curent in main
git remote add origin   -> salveaza adresa repository-ului GitHub sub numele origin
git push -u origin main -> trimite branch-ul main pe GitHub si stabileste legatura implicita
```

Dupa folosirea optiunii -u, commit-urile urmatoare pot fi trimise simplu cu `git push`, fara repetarea numelui `origin` si a branch-ului `main`.

La primul push, se poate deschide browserul pentru autentificare. Autentificati-va si autorizati accesul.

Pasul 8 - Verificarea publicarii

1. Reincarcati pagina repository-ului pe GitHub.
2. Verificati ca fisierele proiectului sunt vizibile.
3. Verificati ca apare commit-ul Lab 01 - proiect Android initial.
4. Confirmati ca repository-ul este Private.

Pasul 9 - Acordarea accesului cadrului didactic

1. Deschideti propriul repository privat, apoi Settings.
 2. In meniul Access, selectati Collaborators si apasati Add people.
 3. Cautati exact alexandra-luca-professor si selectati contul din lista.
 4. Confirmati prin Add alexandra-luca-professor to [numele repository-ului].
 5. Verificati ca invitatia apare pentru contul corect (Pending invite).
 6. Trimiteti linkul repository-ului prin metoda indicata la laborator.
- Repository-ul ramane Private. Accesul devine activ dupa ce cadrul didactic accepta invitatia. Linkul singur nu acorda acces la un repository privat.
- Profil de verificare: <https://github.com/alexandra-luca-professor>

Atentie: pe un repository personal, un collaborator primeste drept de scriere (write), nu doar de citire.

Pasul 10 - Al doilea commit

In panoul Project, apasati clic dreapta pe directorul principal al proiectului si selectati New -> File. Creati fisierul README.md si adaugati descrierea laboratorului. Apoi executati:

```
git status
git add README.md
git commit -m "Lab 01 - adaugare README"
git push
```

Reincarcati pagina repository-ului si verificati ca apare al doilea commit.

20. Varianta prin interfata Android Studio (OPTIONAL DE STIUT)

Aceasta sectiune este doar informativa. Nu trebuie sa executati pasii de mai jos. Pentru laborator folositi comenzile din terminal prezentate in sectiunea 19.

Android Studio poate executa aceleasi operatii Git si prin butoane.

Conectarea contului GitHub in Android Studio

1. Deschideti Settings -> Version Control -> GitHub.
2. Apasati Add account... sau simbolul +.
3. Alegeti autentificarea prin browser.
4. Autentificati-va si autorizati Android Studio.
5. Verificati ca username-ul apare in lista.
6. Lasati debifata optiunea Clone git repositories using SSH.
7. Apasati OK.

Acest pas doar conecteaza contul. Nu creeaza automat un repository.

Operatiile disponibile in interfata

1. VCS -> Enable Version Control Integration -> Git activeaza Git local.

2. Fereastra Commit sau Ctrl+K pe Windows/Linux, respectiv Command+K pe macOS, permite selectarea fisierelor si crearea commit-ului.
3. GitHub -> Share Project on GitHub creeaza repository-ul online si publica proiectul.
4. Git -> Push trimite commit-urile urmatoare.
5. Git -> Pull aduce modificarile existente pe GitHub.

Daca ati urmat deja metoda din terminal pentru proiect, nu mai folositi Share Project on GitHub pentru acelasi repository.

21. Diferenta dintre comenzile Git si butoanele Android Studio

Cele doua coloane realizeaza aceleasi operatii. Pentru laborator folositi coloana Terminal.

Operatie	Terminal - metoda recomandata	Android Studio - varianta grafica
Activarea Git	<code>git init</code>	VCS -> Enable Version Control Integration -> Git
Verificarea modificarilor	<code>git status</code>	Fereastra Commit
Selectarea fisierelor	<code>git add .</code>	Bifati fisierele in Commit
Salvarea unei versiuni	<code>git commit -m "mesaj"</code>	Butonul Commit
Prima publicare	<code>git remote add origin ...</code> si <code>git push</code>	Share Project on GitHub
Trimiterea modificarilor noi	<code>git push</code>	Git -> Push
Aducerea modificarilor online	<code>git pull</code>	Git -> Pull

Nu apasati butonul si apoi executati comanda echivalenta pentru aceeasi operatie.

22. Autentificarea la GitHub

GitHub nu accepta parola contului pentru operatiile Git prin HTTPS.

Pe Windows, Git for Windows include Git Credential Manager, care permite login prin browser.

Pe macOS, instalarea Git prin Command Line Tools nu configureaza automat acest flux.

Daca login-ul prin browser lipseste, instalati GitHub CLI de la <https://cli.github.com>

si rulati `gh auth login`. Alegeti GitHub.com, HTTPS, autentificarea prin browser

si Yes pentru autentificarea Git cu acest cont. Apoi repetati `git push`.

Alternativ, configurati Git Credential Manager conform ghidului oficial GitHub.

23. Fluxul pentru laboratoarele urmatoare

Inainte de a incepe lucrul, executati:

```
git pull
```

Dupa realizarea unei etape, executati:

```
git status
git add .
git commit -m "Lab 02 - descrierea modificarii"
git push
```

Comanda `git add .` selecteaza toate modificarile afisate de `git status`. Pentru a include numai anumite fisiere, folositi `git add fisier1 fisier2`.

Folositi commit-uri progresive. Nu realizati un singur commit cu intregul laborator chiar inainte de termenul-limita.

24. Comenzi Git folosite la laborator

Acestea sunt comenzile pe care le folositi in fluxul recomandat. Android Studio ofera si butoane echivalente, dar nu executati ambele variante pentru aceeasi operatie.

Comanda	Rol
<code>git --version</code>	Verifica instalarea Git
<code>git config --global user.name</code>	Configureaza numele autorului
<code>git config --global user.email</code>	Configureaza e-mailul autorului
<code>git init</code>	Creeaza un repository local
<code>git status</code>	Afiseaza modificarile
<code>git add .</code>	Selecteaza toate modificarile din directorul curent si subdirectoare
<code>git add fisier1 fisier2</code>	Selecteaza numai fisierele indicate
<code>git restore --staged .</code>	Deselecteaza modificarile; necesita un commit existent (vezi pasul 4).
<code>git commit -m "mesaj"</code>	Creeaza un commit local
<code>git branch -M main</code>	Stabileste numele branch-ului principal
<code>git remote add origin URL</code>	Conecteaza repository-ul local la GitHub
<code>git remote -v</code>	Afiseaza adresele remote configurate
<code>git push</code>	Trimite commit-urile pe GitHub
<code>git pull</code>	Aduce modificarile de pe GitHub
<code>git clone URL</code>	Descarca un repository existent
<code>git log --oneline</code>	Afiseaza pe scurt istoricul commit-urilor

Nu folositi comenzi de stergere sau rescriere a istoricului daca nu intelegeti efectul lor.

25. Probleme frecvente Git

Comanda git nu este recunoscuta

- verificati instalarea Git;
- inchideti si redeschideti Android Studio;
- pe Windows, verificati ca Git a fost adaugat in PATH.

Identity unknown

Configurati numele si adresa de e-mail:

```
git config --global user.name "Nume Prenume"
git config --global user.email "adresa-folosita-pe-github"
```

Remote origin already exists

Verificati adresa existenta:

```
git remote -v
```

Nu executati repetat `git remote add origin`.

Autentificarea esueaza

- folositi autentificarea prin browser;
- verificati ca sunteti conectati la contul GitHub corect;

- nu folositi parola obisnuita a contului pentru operatiile Git HTTPS;
- verificati daca invitatia sau accesul la repository este activ.

Au fost adaugate fisiere nedorite

Nu continuati cu push pana nu verificati `.gitignore` si rezultatul comenzii `git status`.

26. Checklist final al laboratorului

```
[ ] Android Studio este instalat
[ ] Android SDK este configurat
[ ] Emulatorul porneste sau telefonul Android este conectat
[ ] Proiectul foloseste Java si XML
[ ] Aplicatia compileaza si ruleaza
[ ] Git este instalat
[ ] Comanda git --version functioneaza
[ ] Numele Git este configurat
[ ] Adresa de e-mail Git este configurata
[ ] Contul GitHub este activ
[ ] Repository-ul este privat
[ ] Fisierul .gitignore exista
[ ] Exista cel putin doua commit-uri
[ ] Commit-urile au fost trimise prin push
[ ] Fisierele sunt vizibile pe GitHub
[ ] Invitatie pentru alexandra-luca-professor este trimisa
[ ] Linkul a fost trimis; accesul este verificat dupa acceptare
```

27. Linkuri utile

- [Descarcare Git](#)
- [Configurarea Git](#)
- [Introducere in GitHub](#)

- [Repository-uri remote](#)
- [Version control in Android Studio](#)
- [Scurtaturi de tastatura in Android Studio](#)

28. Scurtaturi utile in Android Studio

Scurtaturile de mai jos folosesc configurarea implicita a tastaturii din Android Studio.

Actiune	Windows/Linux	macOS
Deschiderea setarilor	Ctrl+Alt+S	Command+,
Deschiderea structurii proiectului	Ctrl+Alt+Shift+S	Command+;
Afisarea panoului Project	Alt+1	Command+1
Afisarea panoului Version Control	Alt+9	Command+9
Deschiderea terminalului	Alt+F12	Option+F12
Cautarea oricarui element	Apasati Shift de doua ori	Apasati Shift de doua ori
Cautarea unei actiuni sau comenzi	Ctrl+Shift+A	Command+Shift+A
Salvarea tuturor fisierelor	Ctrl+S	Command+S
Compilarea proiectului	Ctrl+F9	Command+F9
Rularea aplicatiei	Shift+F10	Control+R
Rularea in modul Debug	Shift+F9	Control+D
Afisarea Logcat	Alt+6	Command+6
Crearea unui commit	Ctrl+K	Command+K
Trimiterea commit-urilor prin Push	Ctrl+Shift+K	Command+Shift+K
Actualizarea proiectului din Git	Ctrl+T	Command+T
Reformatarea codului	Ctrl+Alt+L	Command+Option+L
Sugestie sau corectie rapida	Alt+Enter	Option+Enter
Comentarea/decomentarea unei linii	Ctrl+/*	Command+/*
Cautarea unui fisier	Ctrl+Shift+N	Command+Shift+O
Afisarea fisierelor recente	Ctrl+E	Command+E
Revenirea in editor dintr-un panou	Esc	Esc
Ascunderea tuturor panourilor	Ctrl+Shift+F12	Command+Shift+F12

SDK Manager, Device Manager si Share Project on GitHub nu au, in mod obisnuit, o scurtatura implicita universala. Folositi Find Action, cautati numele actiunii si apasati Enter.

Scurtaturile pot diferi daca a fost schimbat Keymap-ul. Le puteti verifica sau modifica din Settings -> Keymap. Pentru a evita confuzia la laborator, pastrati configurarea implicita.